CPE 464 Lab – Socket API and using poll()

Due:

- Lab worksheet (pdf) – see canvas for due date.
- The lab programming assignment is in a different document – **do this worksheet first**.

Name: Ryan Cramer

Each student must type up their own answers. While you should work as a group, you are required to type up your worksheet and turn in a copy of your work.

I. If you worked in a group, please include your group members' names.

Other Group Members' Names (max of 4 to a group)

II. **Socket API** (Use my sockets lecture and Google to find answers to these questions)

This assumes you have watched the two videos on Canvas.

1. Write the C language command to create a TCP socket for IPv4/IPv6 family?
   socket_n = socket(AF_INET6, SOCK_STREAM, 0);
2. Regarding the bind() function
   a. What information is needed to **name** a socket?

      1. Protocol Family (AF_INET6)
      2. IP Address of Server
      3. Port Number that Server is using

   b. Why do you only need to call bind() on the server? (so why do you need to call it on the server but not on the client)

      You only need to call bind() because the server wants to know what port it's using, and clients know where to connect. The client does not care, so the Operating System takes care of it when the client makes the connect() call.

   c. How does the client get the server's port number?

      It gets the server's port number has to be known before, that's why the server must be set up first before a client can be setup to connect to it.

d. If we do not need to call bind() on the client, how does the client get assigned a port number? (this question is NOT about the server's port number but about the client getting its own port number)

The operating system does it for the client (implicitly) when the client calls connect().

e. How does the server get the client's port number?

It gets the clients port number by filling out the sockaddr struct passed with accept(). After that the server can examine client.sin6_port to know the port number.

3. Regarding the accept() function
   a. When will the accept() function return (remember it's a blocking function)?

   The accept() function will return when a client tries to connect and it's ready to connect. It waits until that happens.

   b. What does the accept() function return (besides -1 on error)?

   It returns a socket file descriptor for communication with that client.

4. When using TCP on the **server** to communicate with clients there are at least two different sockets involved on the **server**. Explain:

   You have to have a main server socket and the client sockets. So one socket is used (the original server socket) that's named with bind() and listens() for connections. The client socket is returned by accept(). That one gets used for send() and recv().

5. Regarding the poll() function (Google man poll):

   a. What does the poll() function do?

   Poll() checks file descriptors and sees if any are ready for I/O or an event has occurred.

   b. Name/explain the parameters passed into the poll() function.

   int poll(struct poll *fds, nfds_t nfds, int timeout);

   The struct poll *fds are all the file descriptors you want to poll.
   The nfds_t nfds is the number of file descriptors.
   The timeout is the timeout to wait in milliseconds.

   c. Regarding the timeout parameter passed to the poll() function
      i. What value will cause the function to block indefinitely?

      -1 will make the poll() function block indefinitely.

ii. What value will cause the function to not block but just look at the socket(s) and return immediately?

0 will cause poll() to not block but look and return.

iii. How can the function be made to block for 3.5 seconds?

You would use a timeout parameter of 3500.

6. How does the server (or client) know that the client (or server) has terminated (e.g. ^C, segfault) when using the recv() call?

If the other side closed the connection, recv() returns 0.
If an error occurred, recv() returns -1.

7. Explain why we need to use the MSG_WAITALL flag on the recv() function call in program #2.

Since TCP is a streaming connection, recv() could not return as many bytes as you wanted from the buffer. But by doing MSG_WAITALL, you force it to grab as many bytes as you requested, unless the connection closes or an error happens.

8. Regarding the code provided with the lab (it is the same code provided with programming assignment #2). The purpose of these questions is to help you understand the code I have provided.

These questions all concern the file: **networks.c**

a. Write the line of code (from network.c) that creates the server_socket:

mainServerSocket = socket(AF_INET6, SOCK_STREAM, 0);

b. What are the line numbers for the code that is used to name the server's socket?

Line 36 sets IPv4/IPv6 family, Line 37 sets in6addr_any, 38 sets htons(serverPort), and 41 bind or names the network. (36-41)

c. What is the effect of the following:

- What does the In6addr_any do (so what does bind() do)?

  (e.g. serverAddress.sin6_addr = in6addr_any;)

  It tells the server to bind to all local IP address to accept connections sent to any of the interfaces.

- If I use a port of 0 in my call to bind(), what does bind() do?

  e.g.

  ```
  serverPort = 0;
  serverAddress.sin6_port= htons(serverPort);
  ```

  If the port is 0, it lets OS pick a specific port.

III. Regarding TCP:

There are several function calls that must be made on a client and server in order to utilize sockets for communications (all in networks.c except send()/recv() and close() which are in myClient.c and myServer.c).

1. In the table below, list the function names (see socket video slide 4) in the order which are needed to setup, use and teardown a connection using Stream Sockets (get this list of functions from the video slides on sockets).
2. In the table below, connect the functions that are part of the connection oriented part of TCP using arrows.
3. Looking at the network.c file provided with this lab put the line number of the call to that function in the table. (Don't worry about line numbers for send()/recv() and close() since these are called in myClient.c and myServer.c.)

Client Line #	**Functions called on the Client**	**Functions called on the Server**	Server Line #
106	socket()	socket()	36
123	connect()	bind()	49
	send()/recv()	listen()	62
	close()	accept()	82
		send()/recv()	
		close()	